

CS370 计算与复杂性

历年卷



Lab8对应2022年的Question3(d),2023年的Question3(d)

Lab7对应2022年的Question3(a)(b)(c),2023年的Question3(c)

Lab7(3)没出现过

Lab6对应2023年的Question2

Lab5对应2023年的Question1(b)(c),2022年的Question2(c)(d)

Lab5(5)没出现过

Lab4对应2022年的Question1、2024年的Question1(a)(c),Question2(a)

Question1

定义

ATM

Let $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$.

HALTTM

$\text{Halt}_{TM} = \{ \langle M, w \rangle \mid \text{TM } M \text{ halts on input string } w \}$.

TM M decides Language L

TM M decides language L :

for every input:

1. M halts on all inputs
2. M accepts all $w \in L$
3. M rejects all $w \notin L$

TM M semidecides Language L

TM M semidecides Language L :

for every input:

1. M accepts all $w \in L$
2. M rejects or loops on all $w \notin L$.

Nondeterministic TM M semidecides a Language L

TM M accepts string w if w is in L , and M either rejects w or does not halt if w is not in L

证明

ATM——可识别性证明 (构造 recognizer)

$U =$ "On input $\langle M, w \rangle$, where M is a TM and w is a string:

1. Simulate M on input w .
2. If M ever enters its accept state, *accept*; if M ever enters its reject state, *reject*."

ATM——不可判定性 (对角线法)

答案内容: Suppose that H is a decider for A_{TM} . On input $\langle M, w \rangle$, where M is a TM and w is a string, H halts and accepts if M accepts w . Formally, we have:

$$H(\langle M, w \rangle) = \begin{cases} \text{accept,} & \text{if } M \text{ accepts } w \\ \text{reject,} & \text{if } M \text{ does not accept } w \end{cases}$$

Now we construct a new TM D , with H as a subroutine. This new TM calls H to determine what M does when the input to M is its own description $\langle M \rangle$. Once D has determined this information, it does the opposite. Formally:

D = "On input $\langle M \rangle$, where M is a TM:

1. Run H on $\langle M, \langle M \rangle \rangle$.
2. Output the opposite of what H outputs; that is, if H accepts, reject; And if H rejects, accept!"

$$\text{i.e. } D(\langle M \rangle) = \begin{cases} \text{accept,} & \text{if } M \text{ does not accept } \langle M \rangle \\ \text{reject,} & \text{if } M \text{ accepts } \langle M \rangle \end{cases}$$

But when we run D with its own description $\langle D \rangle$ as input:

$$D(\langle D \rangle) = \begin{cases} \text{accept,} & \text{if } D \text{ does not accept } \langle D \rangle \\ \text{reject,} & \text{if } D \text{ accepts } \langle D \rangle \end{cases}$$

No matter what D does, it is forced to do the opposite, which is obviously a contradiction. Thus neither TM D nor TM H can exist. $\therefore$ Our assumption is false. $\therefore A_{TM}$ is undecidable.

HALTTM——不可判定性（通常通过 $ATM \leq_m HALTTM$ ）

Assume that TM R decides $HALT_{TM}$. Construct TM S to decide A_{TM} :

S = "On input $\langle M, w \rangle$, an encoding of a TM M and a string w :

1. Run TM R on input $\langle M, w \rangle$.
2. If R rejects, reject.
3. If R accepts, simulate M on w until it halts.
4. If M has accepted, accept; if M has rejected, reject."

Clearly, if R decides $HALT_{TM}$, then S decides A_{TM} . Because A_{TM} is undecidable, $HALT_{TM}$ also must be undecidable.

L decidable $\Leftrightarrow L$ and co- L both recognizable

If both A and $\overline{A}$ are Turing-recognizable, we let M_1 be the recognizer for A and M_2 be the recognizer for $\overline{A}$.

M = "On input w :

1. Run both M_1 and M_2 on input w in parallel.
2. If M_1 accepts, accept; if M_2 accepts, **reject**."

Every string w is either in A or $\overline{A}$. Therefore, Either M_1 or M_2 must accept w . So, M is a decider for A and A is decidable.

Is $\overline{A_{TM}}$ recognizable?

$\overline{A_{TM}}$ is not Turing-recognizable.

PROOF We know that A_{TM} is Turing-recognizable. If $\overline{A_{TM}}$ also were Turing-recognizable, A_{TM} would be decidable. Theorem 4.11 tells us that A_{TM} is not decidable, so $\overline{A_{TM}}$ must not be Turing-recognizable.

Prove the following language is not recursively enumerable

$\overline{H_1} = \{w : \text{either } w \text{ is not the encoding of a Turing machine, or it is the encoding "M" of a Turing machine } M \text{ that does not halt on "M"}\}.$

Assume that H_1 is recursively enumerable. This means there exists a Turing machine M that can semidecide H_1 .

However, by definition of H_1 , " M " $\in H_1$ if and only if M does not accept input string " M ". But M is supposed to semidecide H_1 , so " M " $\in H_1$ if and only if M accepts " M ". The contradiction arises and the assumption is error. Therefore, H_1 is not recursively enumerable.

Question2

定义

Rice's Theorem

Theorem 5.7.4 (Rice's Theorem): Suppose that C is a proper, nonempty subset of the class of all recursively enumerable languages. Then the following problem is undecidable: Given a Turing machine M , is $L(M) \in C$?

Language A is mapping reducible to language B

Language A is mapping reducible to language B , written $A \leq_m B$, if there is a computable function $f : \Sigma^* \rightarrow \Sigma^*$, where for every w ,

$$w \in A \iff f(w) \in B$$

The function f is called the reduction of A to B .

Language A is Turing reducible to language B

- Language A is Turing reducible to language B , written $A \leq_T B$, if A is decidable relative to B .

an oracle TM

An oracle for a language B is an external device that is capable of reporting whether any string w is a member of B .

An oracle Turing machine is a modified Turing machine that has the additional capability of querying an oracle.

State the Church-Turing Thesis

The Church-Turing Thesis posits that any informal **algorithm** is equivalent to **a Turing machine that halts on all inputs**, meaning no computational task beyond such Turing machines can be deemed algorithmic.

Algorithm = Turing Machine that halts on all inputs.

证明

Prove Rice's Theorem

Assume that some TM X decides a property P , and P satisfies the conditions of Rice's theorem. One of these conditions says that TMs A and B exist where $\langle A \rangle \in P$ and $\langle B \rangle \notin P$. Use A and B to construct TM:

R = "On input w :

1. Obtain own description $\langle R \rangle$ using the recursion theorem.
2. Run X on $\langle R \rangle$.
3. If X accepts $\langle R \rangle$, simulate B on w .
If X rejects $\langle R \rangle$, simulate A on w ."

If $\langle R \rangle \in P$, then X accepts $\langle R \rangle$ and $L(R) = L(B)$. But $\langle B \rangle \notin P$, contradicting $\langle R \rangle \in P$, because P agrees on TMs that have the same language.

Therefore, our original assumption is false. Every property satisfying the conditions of Rice's theorem is undecidable.

Show that $ATM \leq_m HALTTM$

(4) We must present a computable function f that takes input of the form $\langle M, w \rangle$ returns output of the form of $\langle M', w' \rangle$, where $\langle M, w \rangle \in A_{TM}$ if and only if $\langle M', w' \rangle \in HALT_{TM}$.

The following machine F computes a reduction f :

F = "On input $\langle M, w \rangle$:

1. Construct the following machine M' : $M' =$ "On input x :
 1. Run M on x
 2. If M accepts, accept.
 3. If M rejects, enter a loop."
2. Output $\langle M', w \rangle$ "

Since we successfully construct this reduction of A_{TM} to $Halt_{TM}$, $A_{TM} \leq_m HALT_{TM}$.

Show that ETM is Turing reducible to ATM (ETM is the emptiness testing problem for TMs).

M_1 = "On input x :

1. If $x \neq w$, reject.
2. If $x = w$, run M on input w and accept if M does."

Assume that TM R decides E_{TM} . Construct TM S that decides A_{TM} as follows.

S = "On input $\langle M, w \rangle$, an encoding of a TM M and a string w :

1. Use the description of M and w to construct the TM M_1 just described.
2. Run R on input $\langle M_1 \rangle$.
3. If R accepts, **reject**; if R rejects, **accept**."

If R were a decider for E_{TM} , S would be a decider for A_{TM} . A decider for A_{TM} cannot exist, so we know that E_{TM} must be undecidable.

Question3

定义

Class P

- P is the class of languages that are **decidable in polynomial time** on a deterministic single-tape Turing machine. In other words, $P = \bigcup_k \text{TIME}(n^k)$.

Class NP

- NP is the class of languages that have polynomial time **verifiers**. In other words, $NP = \bigcup_k \text{NTIME}(n^k)$.

PATH

- $\text{PATH} = \{ \langle G, s, t \rangle \mid G \text{ is a directed graph that has a directed path from } s \text{ to } t \}$

SAT

- $\text{SAT} = \{ \langle \Phi \rangle \mid \Phi \text{ is a satisfiable boolean formula} \}$.
 - A boolean formula is satisfiable if some assignment of 0s and 1s to the variables makes the formula evaluate to 1.

CLIQUE

- $\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique} \}$
 - A clique is a subgraph, where in every two nodes are connected by an edge. A k -clique is a clique that contains k nodes.

HAMPATH

$$HAMPATH = \{ \langle G, s, t \rangle \mid$$

G is a directed graph with a Hamiltonian path from s to t } **Definition:** A Hamiltonian path in a directed graph G is a directed path that goes through **each node exactly once**.

COMPOSITES

$Composites = \{x \mid x = pq, \text{ for integers } p, q > 1\}$ **Definition:** A natural number x is composite if it is the product of two integers greater than 1.

RELPRIME

$RELPRIME = \{ \langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime} \}$ **Definition:** Two numbers are relatively prime if 1 is the largest integer that evenly divides them both.

证明

Show that CLIQUE is in NP

$V =$ "On input $\langle (G, k), c \rangle$:

1. Test whether c is **a set of k nodes** in G .
2. Test whether G contains **all edges** connecting nodes in c .
3. If **both pass**, accept; otherwise, reject."

Show that PATH $\in P$

A polynomial time algorithm M for PATH operates as follows.

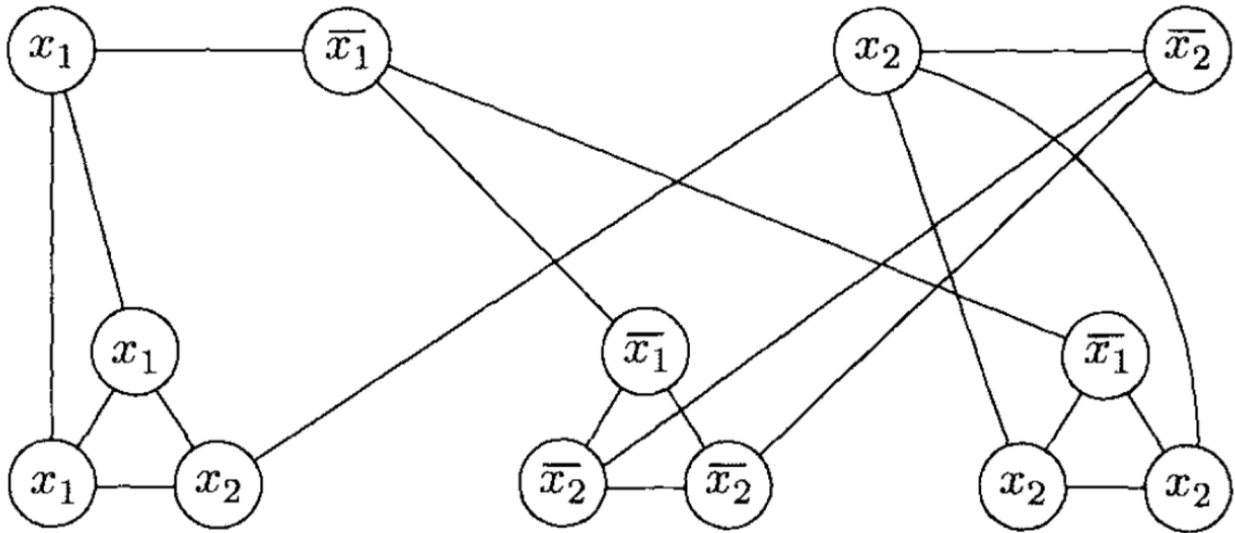
$M =$ "On input $\langle G, s, t \rangle$ where G is a directed graph with nodes s and t :

1. Place **a mark** on node s .
2. Repeat the following until no additional nodes are marked.
3. Scan all the edges of G . If an edge (a, b) is found, going from a marked node a to an unmarked node b , mark node b .
4. If t is **marked**, **accept**. Otherwise, reject."

解释

- (d) Recall the proof 3SAT is polynomial time mapping reducible to VERTEX-COVER ($3SAT \leq_P VERTEX-COVER$). From the following ϕ construct the associated graph G . Explain how to obtain an 8-vertex-cover for G from a satisfying assignment for ϕ .

$$\phi = (x_1 \vee \bar{x}_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$$

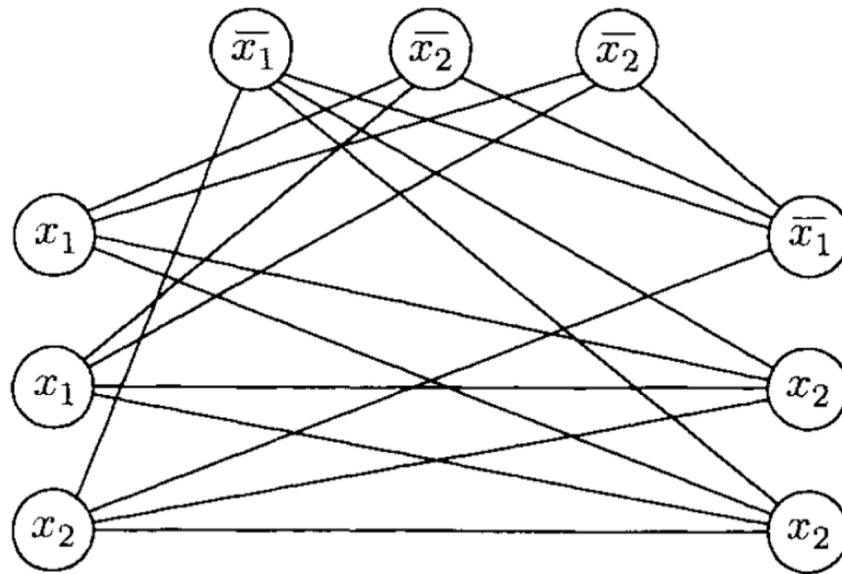


First, put the nodes of the variable that correspond to the true literals in the assignment into the vertex cover (i.e., 2 nodes). Then, select one true literal in every clause and put the remaining two nodes from every clause into the vertex (i.e., $2 \times 3 = 6$ nodes).

Now, we have 8 nodes. They cover all edges because every variable edge is clearly covered, all three edges within every clause are covered, and all edges between variable and clause are covered. Hence, G has an 8-vertex-cover.

- (d) Recall the proof 3SAT is polynomial time mapping reducible to CLIQUE ($3SAT \leq_P CLIQUE$). From the following ϕ construct the associated graph G . Explain how to obtain a 3-clique for G from a satisfying assignment for ϕ .

$$\phi = (x_1 \vee \bar{x}_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$$



First, **select one node corresponding to a true literal in the satisfying assignment**. If more than one literal is true in a particular clause, choose one of the true literals arbitrarily. The number of nodes selected is 3.

Then, **each pair of selected nodes can be joined by an edge** because no pair fits one of the exceptions. They could not be from the same triple because we selected only one node per triple. They could not have contradictory labels because the associated literals were both true in the satisfying assignment.

Hence, G contains a 3-clique.

- (c) Recall the proof 3SAT is polynomial time mapping reducible to VERTEX-COVER ($3SAT \leq_P VERTEX-COVER$). From the following ϕ construct the associated graph G . Explain how to obtain an 8-vertex-cover for G from a satisfying assignment for ϕ .

$$\phi = (\bar{x}_1 \vee x_2 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_2)$$

Lab

(3) Give the definitions of the following classes:

Big-O notation,

Small-O notation,

A verifier,

Big-O notation says that one function is asymptotically no more than another.

In other words, $f(n) = O(g(n))$ means that, for any real number $c > 0$, a number n_0 exists, where $f(n) \leq cg(n)$ for all $n \geq n_0$.

Small-O notation says that one function is **asymptotically less than** another.

In other words, $f(n) = o(g(n))$ means that, for any real number $c > 0$, a number n_0 exists, where $f(n) < cg(n)$ for all $n \geq n_0$.

A verifier for a language A is an algorithm V , where $A = \{ w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c \}$.

(5) Definition: A linear bounded automaton (LBA) is a restricted type of Turing machine wherein the tape head isn't permitted to move off the portion of the tape containing the input. If the machine tries to move its head off either end of the input, the head stays where it is, in the same way that the head will not move off the left-hand end of an ordinary Turing machine's tape.

(a) Proof that: Let M be an LBA with q states and g symbols in the tape alphabet. There are exactly qng^n distinct configurations of M for a tape of length n .

(b) Let $A_{LBA} = \{ \langle M, w \rangle \mid M \text{ is an LBA that accepts input } w \}$. Proof that A_{LBA} is decidable.

(a) Proof: Number of Configurations

A "configuration" is a snapshot of the machine. To find the total number, we multiply the possibilities for each part:

1. **States:** q possibilities.
2. **Head Position:** The head is on one of n tape squares.
3. **Tape Content:** There are n squares, each with g possible symbols (g^n).

$$Total = q \times n \times g^n$$

The Decider:

1. Run the LBA on the input.
2. Count the number of steps it takes.
3. If it runs for more than C steps, it **must** have repeated a configuration.
4. If it repeats a configuration, it is in an infinite loop.
5. **Decision Rule:**
 - If it accepts within C steps: **Accept**.
 - If it rejects within C steps: **Reject**.
 - If it exceeds C steps: Stop and **Reject** (because it's looping).

Since we can set a step limit, we can always decide the answer.